

RoboLink: Build and Drive Your Smartphone-Controlled Robot

MCP: CU-423 (Summer 2026)

Instructor: Mahmoud Sayed and Mahmoud Yassin
Carleton University
Department of Systems and Computer Engineering

Ultrasonic Sensor

In this lab, we will study the working principle of a distance-measurement sensor using ultrasonic waves technology. By the end of this lab you should be able to:

- 1- Describe the working mechanism of the ultrasonic sensor and the distance equation.
- 2- Display and measure the trigger and echo signals on oscilloscope display.
- 3- Trigger the sensor and measure the echo signals using an Arduino.

Ultrasonic Distance Sensor



Introduction

The Ultrasonic distance sensor is used by Remotely Operated Vehicles (ROVs) or Autonomous vehicles to measure distances to obstacles for the purposes of detection and avoidance. Its working principle is similar to Radars: Emit a signal and measure the time it takes to come back again after reflection from the object or obstacle to calculate the distance. The main difference between the Radar and the ultrasonic sensors, from its name, is that the former uses electromagnetic waves while the later uses mechanical (acoustic) waves with 40 KHz in the ultrasonic range (cannot be heard by human ears).

Working Principle

The module does not have a processing unit, which means it does not output the distance measurement as a direct quantity. Instead, it outputs an electric signal that is proportional to the measured distance. To understand how it works, let's have a look at its 4 pins.

- **Vcc:** The power pin. Should be connected to the DC supply.
- **Trig:** The trigger pin is an input to the sensor that is driven by the microcontroller to initiate a measurement process.
- **Echo:** The echo pin is an output from the sensor. The sensor outputs a signal on this pin that is proportional to the measured distance.
- **Gnd:** The ground pin. Should be connected to the circuit ground.

The timing diagram of the sensor signals is illustrated in Fig. 1.

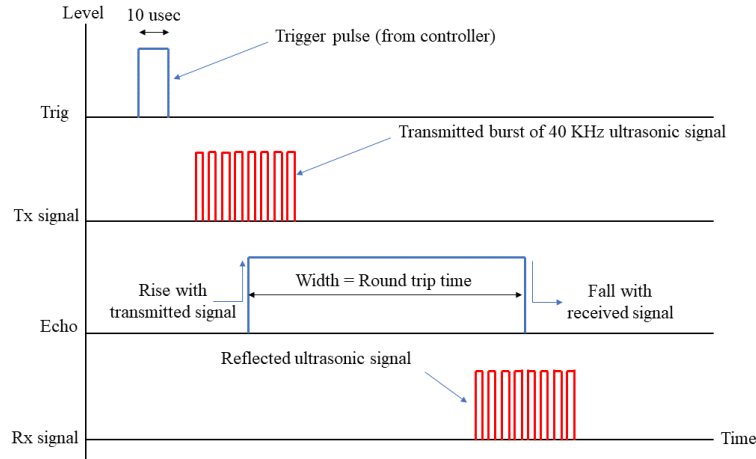


Fig. 1 Timing diagram of the ultrasonic sensor's signals

As can be seen from Fig.1, the measurement process starts by sending a 10 usec pulse from the microcontroller to the Trig pin. Then, the sensor begins transmitting a burst of 40 KHz ultrasonic pulses from its Tx terminal and it also raises the voltage on the Echo pin at the same time. When the reflected ultrasonic is received at the Rx terminal, the sensor pulls the voltage on the Echo pin to the Gnd voltage. Hence, the width of the pulse generated on the Echo pin represents the time taken by the ultrasonic signal to travel from the Tx to an obstacle and back to the sensor again i.e., the roundtrip time. Knowing that the speed of sound in air is approximately 343 m/sec, if the echo pulse width is T_p then the distance to the obstacle D can be calculated as:

$$D = 343 \times \frac{T}{2} \text{ (m)} \quad (1)$$

The division by 2 is required since T is the round-trip time. Table 1 gives a summary of the sensor specifications:

Table 3: VMA330 module specifications.

Operating Voltage	3.3V to 5V
Acoustic frequency	40 KHz
Measurement Angle	15°
Range (claimed)	5.5 m
Trigger format	20 usec pulse.
Output type	Pulse width modulated output
Working temperature	-10 to +60 °C

Some Practical Considerations

For the sensor to operate properly, the following points should be carefully considered.

- 1- Trigger signal:** The trigger signal should be a rectangular pulse with minimum recommended width of 10 usec. Each trigger pulse will initiate a single measurement cycle. So, repeated pulses are required for repeated measurement cycles i.e. continuous measurement. In this case, the trigger signal will be then as a short duration periodic pulse or, equivalently, a pulse width

modulated signal with low duty cycle. The minimum width of the pulse is specified by the datasheet to be 10 usec, but what about the period? What should be the distance between two consecutive pulses? To answer this question, we can assume that the sensor will work at its full range of 5.5m. Before triggering the sensor for the cycle n , we should wait to make sure that cycle $n - 1$ is completely finished by giving the ultrasonic signal enough time to reach the maximum distance and come back again. Assuming the maximum distance is 5.5 m, then the total round-trip distance is 11 m and the maximum round-trip time T_{max} would be:

$$T_{max} = \frac{11m}{340m/sec} = 32.35 \text{ msec} \quad (2)$$

This means that the time spacing between two consecutive pulses should not be less than 32.35 msec, or the trigger pulses frequency should be more than 30.91 Hz. You may need to increase the period more than that to accommodate for another practical issues, it is advised for some ultrasonic sensors that the period be 60 msec at least.

- 2- **Sensor Orientation:** The sensor has a measurement angle of 15° , which can cause erroneous readings if the sensor is placed very close to the ground because of unwanted reflection from the ground surface.
- 3- **Echo Signal:** The width of the echo signal should be measured in order to calculate the distance. Measuring time depending on the microcontroller's CPU solely will cause blocking behavior: The CPU will be stuck while measuring the time and won't be able to do anything else, including monitoring other sensors for example, till the measurement process ends. Usually, this is unwanted behavior. Most microcontrollers have modules called "timers" that are dedicated to the timing tasks. The SAM3X8E microcontroller of the Arduino Due is indeed full of resources and has 9 timer modules with massive number of options. You may need to utilize these timers to measure the time to avoid blocking the CPU.

Experiment 1: Test the Ultrasonic Distance Module using Oscilloscope.

In this experiment, you should test the basic functionality and performance of the ultrasonic distance sensor module.

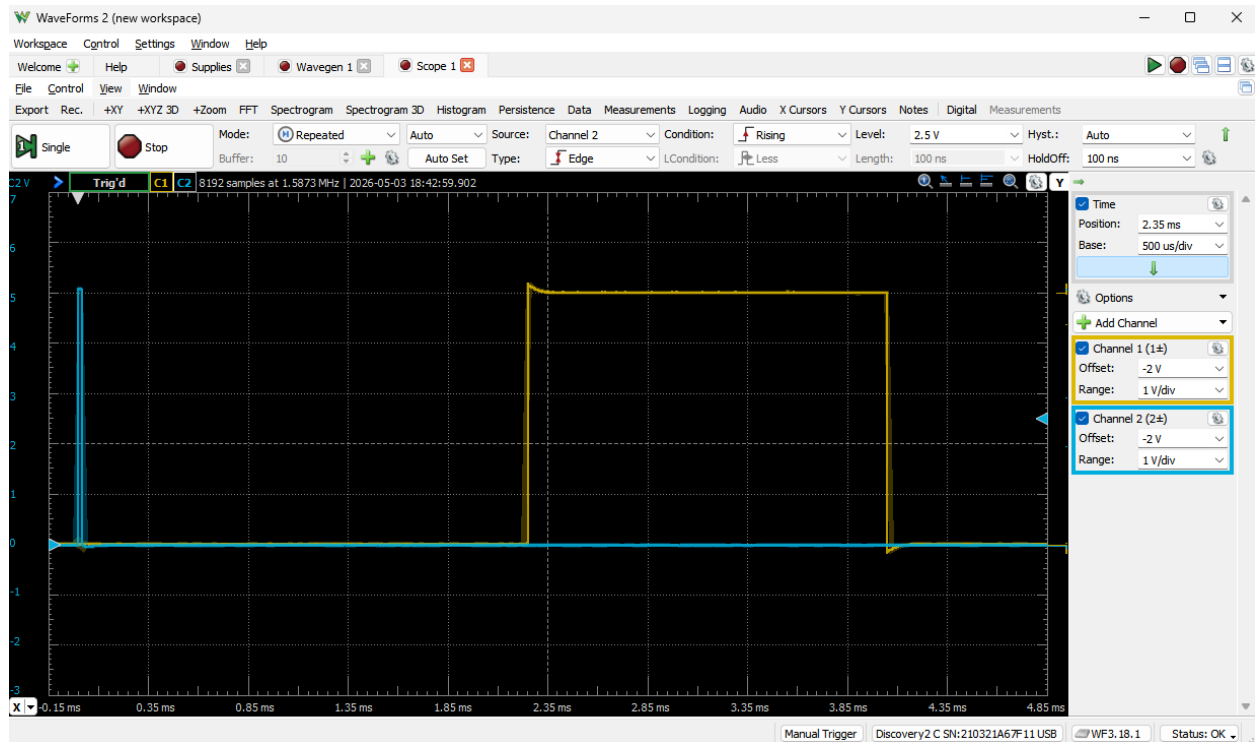
Required Equipment:

- 1- Ultrasonic distance sensor module.
- 2- Oscilloscope (Analog Discovery 2)
- 3- Breadboard.
- 4- Jumper wires.

Procedure:

- 1- **Configure the AD2 power supply:** We will use the voltage supply of the AD2 to power the sensor.
 - a. Go to "Supplies" tab in the Waveforms software
 - b. Configure the positive supply to generate 5V
 - c. Connect the pins "V+" and "GND" as your power terminals to power the sensor, so connect them to the pins Vcc and GND of the sensor on the breadboard.
 - d. Do not turn on the power supply until everything else is connected and ready.
- 2- **Configure the Trigger signal:** We will use the AD2 to generate the trigger signal.

- a. Go to the “Wavegen” tab
 - b. Select “Channel 1” (Next to the “Run All” button)
 - c. Type: Square Wave
 - d. Period: 50 ms
 - e. Amplitude: 2.5V
 - f. Offset: 2.5V
 - g. Symmetry: 0.05% (You can type it).
 - h. Connect W1 pin of the AD2 to the “Trig” pin of the sensor.
- 3- Configure the scope to draw the echo signal. We will use the Scope to draw the trigger and echo signals of the sensor.
 - a. Connect the pins “+1” and “-1” of the AD2 to the “Trig” and “Gnd” pins of the sensor.
 - b. Connect the pins “+2” and “-2” to the AD2 to the “Echo” and “Gnd” pins of the sensor.
 - c. Go to “Scope” tab in the Waveforms software.
 - d. Time configurations:
 - i. Position: 2.35 msec
 - ii. Base: 500 us/div
 - e. Channel 1 & Channel 2
 - i. Offset -2V
 - ii. Range: 1V/Div
 - f. Source (Trig)(in the upper panel): Channel 1
 - g. Condition: Rising
 - h. Level: 2.5V



- 4- **Check the operation:** Move an obstacle in front of the module to check its functionality. You should notice changing in the width of the echo pulse; as the obstacle gets further from the sensor, the pulse echo pulse width increases and vice versa.
For testing, note that, if there is no obstacle at all, the echo signal should still show a rectangular square wave with around 32.35 msec of HIGH. If you can't see a rectangular waveform, please recheck the connection, and/or inform your instructor.
- 5- **Measure the distance:** With the sensor functioning as expected, it is time to take detailed measurements. Adjust the obstacle distance from the sensor according to Table 4 and measure the pulse width at each distance, using the "Measure" functionality of your Oscilloscope. Compare it to the actual distance and calculate the error. What is the minimum and maximum detectable distances?

Table 4: Experiment 1 Measurements

Actual Distance (cm)	Using Oscilloscope			Arduino Due		
	Pulse width	Measured Distance	Distance Error	Pulse width	Measured Distance	Distance Error
min						
10						
20						
30						
max						

Experiment 2: Test the Ultrasonic Sensor using Arduino.

In this experiment, you should interface the ultrasonic module with the Arduino board to provide trigger and to measure the echo. The Arduino board should utilize the ultrasonic module to measure the distance to an obstacle and display it on the serial monitor.

Required Equipment:

- 1- Arduino Nano board.
- 2- Ultrasonic sensor module.
- 3- Breadboard.
- 4- Jumper Wires.

Using `PulseIn()` to measure, and using `delayMicroseconds()` to trigger.

- 1- **Make the connections:**
 - Connect the Arduino Nano to an empty location on the breadboard.
 - Use Jumper wires to connect the 5V and GND pins from the Arduino to the power strips on the breadboard.
 - Connect the Ultrasonic Sensor to an empty location on the breadboard, and make the following connections.

- 5V power line -> Vcc of the sensor
 - GND line -> GND of the sensor
 - Arduino pin D2 -> Trig pin of the sensor
 - Arduino pin D3 -> Echo pin of the sensor
- 2- **Write your code:** Develop an Arduino sketch that does the following in a loop:
- In the setup function:
 - Make pin 2 as output (It outputs the trigger signal from the Arduino to the Sensor)
 - Make pin 3 as input (it reads back the echo signal from the sensor)
 - Initialize the Serial “Serial.begin(115200);”
 - In the loop function:
 - The Arduino applies the Trigger signal by deriving pin 2 to HIGH.
 - Wait for 50 usec using delayMicroseconds().
 - Turn off the TRIG_PIN by deriving it to Low.
 - Poll pin 3 using the Arduino function “pulseIn(3,HIGH)” to measure its pulse width.
 - Calculate the distance using the measured pulse width time according to equation (1). Do not forget that the measured time using pulseIn() is in microseconds.
 - The Arduino should send the reading value to the computer using the serial terminal to be displayed on the serial monitor, along with your group number!
 - Wait for 500 msec using delay(500) then repeat the loop.
- 3- **Take measurements:**
- As done in Experiment 1, move the obstacle according to the distance values in Table 2 and fill in the pulse width, calculate the distance, and calculate the error (Complete third section in Table 2).

Sample Code:

```
void setup() {
  // put your setup code here, to run once:
  pinMode(2, OUTPUT); // The trigger pin.
  pinMode(3, INPUT);  // The echo pin.
  // Intitalize the Serial transmitter
  Serial.begin(115200);
}

void loop() {
  // put your main code here, to run repeatedly:
  //Send out the trigger signal
  digitalWrite(2, HIGH);

  //Wait for 10 microseconds
  delayMicroseconds(50);

  //Lower the trigger pin
  digitalWrite(2, LOW);
```

```
//Measure the distance (The time at which the echo signal is HIGH)
unsigned long int distance_in_time = pulseIn(3, HIGH);

//Convert time to distance
// Divide by 2 because of the round-trip
// Divide by 10,000 to convert from micrometers to centimeters (the time is
measured originally in microseconds)
// We are converting to float to get more resolution.
float distance_in_cm = 343 * distance_in_time / (2*10000.0);

Serial.print("Distance: ");
Serial.println(distance_in_cm);
delay(500);
}
```